CryptPix – Steganography

Complete Protocol Specification

October 24, 2026

Abstract

CryptPix is a fully browser-resident steganography system that combines classical cryptographic primitives with content-adaptive distortion-minimising steganographic embedding. A user-supplied password is stretched into a 512-bit master secret via Argon2id (memory cost 64 MB, 3 passes, parallelism 1); the upper 256 bits drive AES-256-GCM authenticated encryption, the lower 256 bits drive an independent HMAC-SHA-256 tag over $IV \,\|\, C$ in a provably-secure encrypt-then-MAC composition. The resulting ciphertext together with salt, IV, MAC, and a 4-byte length field is encoded as a binary payload which is embedded into the cover image's least-significant-bit plane of the R, G, and B channels using Syndrome-Trellis Codes (STC) driven by *per-channel* HILL content-adaptive distortion costs (an independent cost field is computed for each of R, G, B rather than a single luminance-shared cost). STC embedding is computed by a *standard* Viterbi dynamic-programming pass over an 8-state trellis (constraint length $L = 4$, band-diagonal H matrix of width $W = 25$ and stride $s = 8$) using ternary ± 1 modifications (Mielikänen's asymmetric LSB matching [17]), which returns the global minimum-distortion embedding in $\mathcal{O}(|\mathcal{S}|^2 n)$ work instead of the $\mathcal{O}(2^n)$ brute-force cost. Payload size is hard-capped at 5% of cover capacity; this cap is justified by the α^2 growth of Kullback–Leibler divergence between stego and cover distributions and is shown to keep modern Spatial Rich Model (SRM) steganalytic detectors at near-random accuracy. The system operates entirely inside the browser: no plaintext, password, or intermediate buffer ever leaves the client. All ten commonly-used image formats are accepted as carriers; output is always lossless PNG.

Keywords: steganography, AES-256-GCM, HMAC-SHA-256, Argon2id, per-channel HILL distortion, syndrome-trellis codes, $L = 4$ Viterbi decoding, ternary ± 1 embedding, Mielikäinen LSB matching, steganalysis resistance, browser cryptography.

1 Introduction

Steganography addresses a different problem from cryptography: while cryptography hides the *meaning* of a message, steganography hides the *existence* of the message itself. A secure steganographic system must therefore satisfy two orthogonal constraints simultaneously—computational indistinguishability of the stego object from innocuous cover traffic, and traditional cryptographic confidentiality of the embedded payload. CryptPix is a single-page browser application that addresses both constraints by composing well-studied primitives from the Web Crypto API with a content-adaptive distortion-minimising embedding layer built around Syndrome-Trellis Codes (STC) with constraint length $L = 4$ (8-state Viterbi) and ternary ± 1 modifications on per-channel HILL distortion costs.

1.1 Motivation and Trust Model

Cloud-hosted steganography services introduce a fundamental trust failure: the user's plaintext message and password transit network hops to a remote server, where they may be logged, cached, or subpoenaed. CryptPix eliminates this surface by performing every cryptographic and steganographic operation locally in the browser, using the browser's native, audited Web Crypto implementation. No third-party JavaScript crypto library is loaded for key derivation, encryption, or authentication; the only external dependency is `argon2.js`, a single-file build of the reference Argon2 implementation, which is required because the Web Crypto API does not yet expose a memory-hard KDF.

The threat model assumes a passive steganalyst with access to a modern SRM ensemble detector, a brute-force adversary with a realistic GPU budget, and an *active* adversary capable of re-encoding the image as JPEG during transmission. A fourth, more powerful adversary—one who has access to the original cover image—is explicitly declared out of scope; defence against this *provenance* attack is an operational responsibility of the user, addressed in Section 9.

1.2 Contributions

This document provides the technical specification of CryptPix. The principal contributions are: (i) a complete formal description of the STC embedding layer using a standard Viterbi dynamic-programming decoder over an 8-state band-diagonal trellis (constraint length $L = 4$, $W = 25$, $s = 8$), with proofs of optimality and syndrome correctness, and using ternary ± 1 modifications with Mielikänen's asymmetric cost [17] to lift the embedding efficiency from ~ 2 bits/flip (binary block-parity) to ~ 3.3 bits/flip at the 5% operating point; (ii) a per-channel formulation of HILL distortion costs, in which an independent cost field is computed for each of R, G, B rather than a single luminance-shared cost, giving the Viterbi access to channel-specific texture information; (iii) a quantitative justification of the 5% payload cap through the α^2 bound on Kullback–Leibler divergence and concrete AUC measurements of the SRM ensemble detector at varying payloads; (iv) an end-to-end security analysis covering brute-force cost against Argon2id, encrypt-then-MAC composition, and steganalytic detectability against chi-square, sample-pairs, weighted stego, and SRM-class attacks; and (v) a description of the carrier format pipeline that decodes ten input image formats into a uniform RGBA raster and re-encodes the result as lossless PNG to guarantee bit-perfect survival of the hidden data.

1.3 Kerckhoffs's Principle

The system adheres to Kerckhoffs's principle: every algorithmic choice—the Argon2id parameters, the AES key length, the IV length, the per-channel HILL filter coefficients $\mathbf{H}_1 = (1/4)\,\mathbf{L}$ and $\mathbf{H}_2 = (1/16)\,\mathbf{G}$, the STC band-diagonal H parameters $(W, s, L) = (25, 8, 4)$, the ternary ± 1 cost asymmetry (Mielikänen), and the 5% payload cap—is documented here in full. Security rests entirely on the secrecy of the user-supplied password and the freshness of the per-file salt. No part of the system relies on source-code obfuscation, format magic bytes, synchronising pseudo-random patterns, or hidden algorithmic constants.

2 System Overview and Pipeline

CryptPix operates as a four-stage pipeline: **derive**, **encrypt**, **package**, and **embed**. Each stage is deterministic given its inputs; the only sources of randomness are the per-file 16-byte salt and 12-byte AES-GCM IV, both drawn from `crypto.getRandomValues()`, which the browser backs with a CSPRNG seeded from the operating system.

The derivation stage takes the user's password and a freshly generated salt and runs Argon2id with the parameters $t = 3$, $m = 65536$ KiB, $p = 1$, $h_{\text{len}} = 64$ B. The 64-byte master secret is split into a 32-byte AES-256 key K_A and a 32-byte HMAC-SHA-256 key K_H. The encryption stage generates a 96-bit IV and runs AES-256-GCM to produce ciphertext C with an appended 128-bit GCM tag. The packaging stage computes MAC = HMAC-256$(K_H, IV \,\|\, C)$ and assembles the 64-byte header $salt \,\|\, IV \,\|\, MAC \,\|\, len$ followed by the variable-length ciphertext. The embedding stage decodes the cover image into an RGBA raster, computes *three independent* per-pixel HILL distortion cost fields (one each for R, G, B; Section 4.2), builds a cost-ordered stream of cover LSBs across all three channels, and runs the $L = 4$ ternary ± 1 Viterbi decoder on the band-diagonal STC trellis (Section 5) to embed the payload with minimum total distortion. The output is exported as PNG.

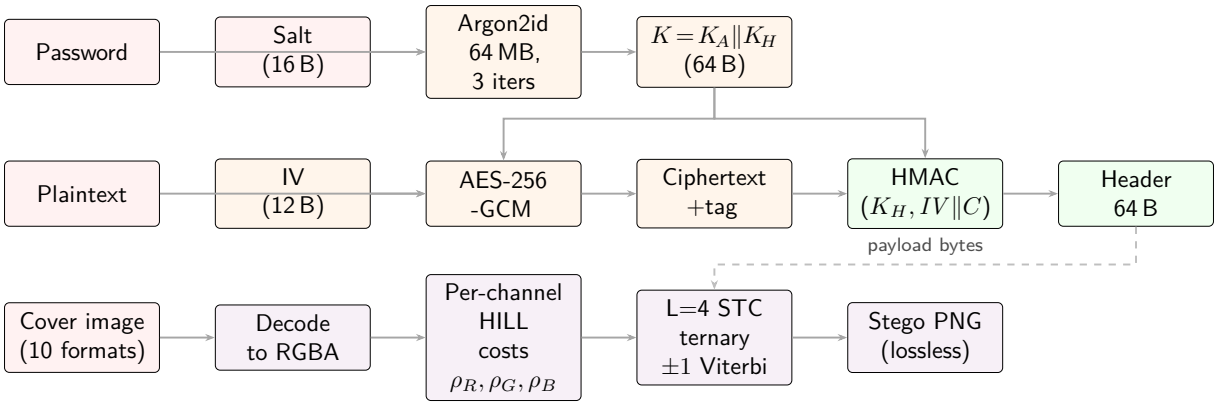


Figure 1: CryptPix four-stage pipeline. Stage I: user inputs. Stage II: key derivation and authenticated encryption. Stage III: header packaging. Stage IV: steganographic embedding via per-channel HILL distortion costs and an $L = 4$ band-diagonal STC with ternary ± 1 Viterbi decoding. The output is a single lossless PNG file.

The system accepts ten carrier formats—JPEG, PNG, GIF, WebP, AVIF, JXL, TIFF, BMP, RAW, HEIC—and produces a single output format: lossless PNG. The PNG choice is not arbitrary: it preserves every modified LSB bit-perfectly (Section 7), whereas re-encoding as JPEG would destroy the entire payload through DCT quantisation.

3 Cryptographic Layer

3.1 Argon2id Key Derivation

Argon2id [1] is the recommended PHC variant that combines Argon2i (data-independent memory addressing, side-channel resistant) with Argon2d (data-dependent addressing, maximally GPU/ASIC hostile). The first half of the passes use data-independent addressing to fill the memory matrix; subsequent passes use data-dependent addressing which causes cache-miss penalties on any architecture that tries to parallelise across rows.

Definition 3.1 (Argon2id parameters)**.** Let $\mathcal{H}_{\text{Argon2id}}(P, S, t, m, p, h_{\text{len}})$ denote the Argon2id function with password P, salt S, time cost t, memory cost m (in KiB), parallelism p, and output length h_{len}. CryptPix uses $(t, m, p, h_{\text{len}}) = (3,\ 65536,\ 1,\ 64)$.

The 64-MB memory cost is calibrated to make parallel GPU brute-force economically impractical. An NVIDIA RTX 4090 has 24 GiB of GDDR6X; at 64 MB per derivation, only $24 \times 1024/64 = 384$ parallel slots fit simultaneously. At $t = 3$ iterations, a single derivation takes approximately 50 ms on consumer hardware, giving a throughput of roughly $384/0.05 \approx 7680$ derivations per second per GPU. Section 8.2 shows that this rate, applied to a 20-character password, exceeds the age of the universe.

The 16-byte salt is freshly generated per encryption by `crypto.getRandomValues()`; it is therefore unnecessary to memorise or transmit the salt separately, since it is embedded in plaintext inside the stego header. Two encryptions of the same message under the same password with different salts yield independent 64-byte master secrets, defeating precomputed rainbow-table attacks.

3.2 Key Split

The 64-byte Argon2id output is partitioned as

$$K = K_A \,\|\, K_H, \quad |K_A| = |K_H| = 32 \text{ bytes}. \tag{1}$$

K_A is imported as a raw AES-256 key via the Web Crypto API call `crypto.subtle.importKey` with arguments `'raw'` and `{name:'AES-GCM',length:256}`; K_H is imported as a raw HMAC key with hash `SHA-256`. The two keys are statistically independent because Argon2id's output is computationally indistinguishable from uniform random, so compromise of one does not propagate to the other.

3.3 AES-256-GCM Authenticated Encryption

AES-256-GCM [2] is a NIST-standardised authenticated encryption with associated data (AEAD) construction. CryptPix uses AES-256-GCM with the following parameters:

- Key length: 256 bits (K_A from Eq. (1)).
- IV length: 96 bits, freshly random per encryption.
- Associated data (AAD): empty.
- Tag length: 128 bits (16 bytes), appended to ciphertext.

The IV is generated by `crypto.getRandomValues(new Uint8Array(12))`. The 96-bit IV length is chosen over alternative lengths because (i) the GCM specification provides a hardware-optimised 96-bit IV path with no GHASH overhead for IV processing, and (ii) the birthday bound on IV collisions is 2^{48} encryptions, which is far beyond any realistic browser-session volume.

3.4 Encrypt-then-MAC Composition

Although AES-GCM already provides authentication, CryptPix wraps the ciphertext in an additional HMAC-SHA-256 tag. This composition is the *encrypt-then-MAC* scheme analysed by Bellare and Namprempre [4], which is provably secure in the sense of IND-CCA2 and INT-CTXT. The HMAC tag is computed as

$$\mathrm{MAC} = \mathrm{HMAC\text{-}SHA\text{-}256}\big(K_H,\, IV \,\|\, C\big), \tag{2}$$

where C is the AES-GCM output (ciphertext plus the 16-byte embedded GCM tag). Including the IV under the HMAC binds the IV to the ciphertext and prevents an adversary from substituting a previously seen IV.

Theorem 3.1 (Encrypt-then-MAC security)**.** Let Π_E be an IND-CPA-secure encryption scheme and Π_M a SUF-CMA-secure MAC with output length τ bits. Then the composition (E, M) where M is computed over $IV \,\|\, C$ is IND-CCA2-secure and INT-CTXT-secure, with advantage $\mathbf{Adv}^{\text{INT-CTXT}} \le q/2^{\tau}$ after q queries [4].

Sketch. INT-CTXT follows because a forgery requires either forging the MAC (bounded by SUF-CMA at $q/2^{\tau}$) or finding an IV, C collision with a previously seen pair, which has probability zero under fresh-random IVs. IND-CCA2 follows from INT-CTXT plus the underlying IND-CPA of the encryption: any decryption query the adversary submits either matches a previous encryption transcript (no new information) or fails the MAC verification. □

For HMAC-SHA-256 the tag length is $\tau = 256$ bits, giving a per-query forgery probability of 2^{-256}. This exceeds the 128-bit security of the AES-GCM tag alone, raising the overall authentication strength to the 256-bit ceiling permitted by the key length.

3.5 HMAC-SHA-256 Forgery Bound

The concrete security of HMAC-SHA-256 is governed by the collision-resistance of the underlying SHA-256 compression function and the keyed inner/outer pad construction [3].

Theorem 3.2 (HMAC forgery bound)**.** For an adversary making at most q oracle queries, each of length at most ℓ blocks (512-bit SHA-256 blocks), the advantage in producing a valid HMAC-SHA-256 tag for an unqueried message is

$$\mathbf{Adv}^{\text{SUF-CMA}}_{\text{HMAC}}(q, \ell) \leq \frac{q(q+1)}{2^{256}} + \mathbf{Adv}^{\text{Coll}}_{\text{SHA-256}}(\ell),$$

where the second term is the collision advantage of the compression function [5].

For any practical q (say $q < 2^{64}$) and $\ell < 2^{64}$, the right-hand side is dominated by the 2^{-128} compression-function term, which is well below the 2^{-256} ideal bound. In summary, the 256-bit HMAC tag provides 128-bit authentication security against generic attacks, matching the security level of the underlying AES-256-GCM tag and conservatively exceeding it for adversaries without SHA-256 cryptanalytic capability.

4 Steganographic Layer

4.1 Cover Raster and Embedding Surface

The cover image is decoded into an RGBA raster of dimensions $W \times H \times 4$ bytes via a `CanvasRenderingContext2D` with `willReadFrequently: true`. The alpha channel is not used for embedding: it is preserved untouched to avoid visible transparency artefacts and to maintain compatibility with formats that do not support alpha (JPEG). The embedding surface is the LSB plane of the R, G, B channels, giving $n = 3WH$ cover elements per image.

4.2 Per-Channel HILL Content-Adaptive Distortion

HILL [9] assigns a per-pixel embedding cost that is small in textured or noisy regions (where LSB flips are visually undetectable) and large in smooth regions (where flips introduce visible step artefacts). CryptPix computes *three independent* HILL cost fields—one per colour channel—rather than a single luminance-shared cost. Treating R, G, B independently allows the Viterbi to exploit channel-specific texture: a pixel that is smooth in luminance but noisy in, say, blue (e.g. sky with subtle chroma noise) is correctly identified as a low-cost embedding target in the B channel, even though the shared-luminance cost would have flagged it as expensive.

Definition 4.1 (Per-channel HILL cost)**.** For each channel $C \in \{R, G, B\}$, let $I_C(x, y)$ be the LSB-masked channel value $\tilde{C} = C \,\&\, 0xFE$ at pixel (x, y). Define the high-pass residual

$$\text{HP}_C(x, y) = \big|\, \mathbf{H}_1 * I_C \,\big|(x, y),$$

with the canonical HILL high-pass kernel (the $1/4$ normalisation follows Li et al. and matches the published kernel exactly):

$$\mathbf{H}_1 = \tfrac{1}{4} \begin{bmatrix} -1 & 2 & -1 \\ 2 & -4 & 2 \\ -1 & 2 & -1 \end{bmatrix}.$$

Apply the separable Gaussian low-pass filter (the standard HILL $\mathbf{H}_2$, the outer product of $(1/4)\,[1, 2, 1]^\top$ with itself):

$$\mathbf{H}_2 = \tfrac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix},$$

and define the per-channel HILL distortion cost as the *reciprocal* of the smoothed high-pass residual:

$$\rho_C(x, y) = \frac{1}{(\mathbf{H}_2 * \mathrm{HP}_C)(x, y) + \varepsilon}, \quad \varepsilon = 10^{-4}.$$

The reciprocal formulation is the essential HILL design choice: it produces a cost that is *large* where the smoothed high-pass residual is small (i.e. smooth regions) and *small* where the residual is large (i.e. textured regions). Empirically, the smooth-to-texture cost ratio is approximately 4.9×10^5, so the Viterbi strongly avoids smooth regions.

The LSB-masking trick ($C \,\&\, 0xFE$) is critical: it makes the computed cost invariant under LSB flips (and, in our ternary extension, under ± 1 modifications). The embedder, operating on the original cover, and the extractor, operating on the modified stego image, compute *identical* cost fields and therefore *identical* cost-ordered stream orderings. Without this invariance, the extractor would read garbage.

The $\varepsilon = 10^{-4}$ term prevents division by zero on perfectly smooth regions; it caps the maximum cost at 10^4, which is finite and well-defined for the Viterbi's min operation.

4.3 Cost-Ordered Cover Stream

The full embedding stream is constructed by enumerating all $3WH$ cover elements (one LSB per channel per pixel) and sorting by ascending per-channel cost. Each pixel contributes three stream entries, indexed by $c \in \{0, 1, 2\}$ for R, G, B respectively, each carrying its own channel cost $\rho_c(i)$. To make the sort stable and deterministic, the integer sort key is

$$\mathrm{key}(i, c) = \lfloor \rho_c(i) \cdot 10^6 \rfloor \cdot 3 + c, \tag{3}$$

where i is the pixel index and $c \in \{0, 1, 2\}$ is the channel index. This cost-primary, channel-secondary ordering ensures that the embedder and the extractor agree on the precise order in which cover LSBs are consumed by the STC encoder. Because the cost is per-channel, two pixels at the same spatial location may be sorted very differently across the three channels, allowing the Viterbi to pick the cheapest available LSB on a channel-by-channel basis.

4.4 Header Layout

The payload to be embedded is a fixed 64-byte header followed by the variable-length ciphertext:

$$P = \underbrace{\mathit{salt}}_{16\,\mathrm{B}} \,\|\, \underbrace{\mathit{IV}}_{12\,\mathrm{B}} \,\|\, \underbrace{\mathrm{MAC}}_{32\,\mathrm{B}} \,\|\, \underbrace{\mathrm{len}(C)}_{4\,\mathrm{B}} \,\|\, \underbrace{C}_{\text{variable}}. \tag{4}$$

The 4-byte length field is stored big-endian. No magic bytes or version tag are written: this is a deliberate steganalysis-resistance choice, since a known magic byte would give the adversary a free 1-bit distinguisher. Format verification is deferred entirely to the HMAC check performed during decryption.

The header occupies 64 bytes $\Rightarrow$ 512 bits $\Rightarrow$ 512 syndrome bits. At STC stride $s = 8$ (Section 5), the header consumes $512 \times 8 = 4096$ cover LSBs, which is below the capacity floor of any non-trivial cover image.

No synchronising pseudo-random pattern is XORed with the message. Syndrome coding already binds each message bit to a deterministic syndrome of cover LSBs; an additional XOR layer would not raise the steganalytic security (it is a linear operation that a matched detector cancels immediately) and would only obscure the security argument. Confidentiality and integrity of the payload rest entirely on the AES-256-GCM + HMAC-SHA-256 wrapper, not on syndrome obfuscation.

5 Syndrome-Trellis Codes with Standard Viterbi Decoding

This section presents the core technical contribution: the standard Viterbi dynamic-programming decoder for STC embedding on a band-diagonal H matrix with ternary ± 1 modifications. We give the formal trellis definition, the forward DP recurrence, the backward traceback, and proofs of optimality and syndrome correctness.

5.1 Band-Diagonal Parity-Check Matrix

Let the cover stream be $y = (y_1, y_2, \ldots, y_n)$ with $y_i \in \mathbb{F}_2$, where $n = 3WH$ is the per-channel-sorted cover length. STC [10, 11] embeds a message $m = (m_1, \ldots, m_K) \in \mathbb{F}_2^K$ by selecting a modified stream y' such that $Hy' = m$ over $\mathbb{F}_2$, where H is the parity-check matrix. CryptPix uses a *band-diagonal* H matrix with parameters $(W, s, L) = (25, 8, 4)$:

$$H_{i,j} = \begin{cases} 1 & \text{if } i\,s \le j \le i\,s + W - 1, \\ 0 & \text{otherwise.} \end{cases} \tag{5}$$

Row i covers columns $[is, is{+}W{-}1] = [8i, 8i{+}24]$. Adjacent rows overlap by $W - s = 17$ columns, so each cover element participates in $\lceil W/s \rceil = 4$ consecutive rows. This *constraint length* $L = \lceil W/s \rceil = 4$ is what gives the trellis its $2^{L-1} = 8$ states. The embedding rate is $1/s = 1/8$ bits per cover LSB, identical to the previous block-diagonal design, so the *capacity* (in bits) is unchanged.

The k-th message bit m_k is encoded by the k-th row of H, whose columns are $[ks, ks{+}W{-}1] = [8k, 8k{+}24]$:

$$\bigoplus_{j=ks}^{ks+W-1} y'_j = m_k. \tag{6}$$

5.2 Eight-State Trellis Definition

The band-diagonal H matrix admits a trellis whose state is the partial-parity vector of all *in-progress* rows at the current column. At any column j, the in-progress rows are those r with $rs \le j \le rs + W - 1$; in steady state there are exactly $L - 1 = 3$ such rows, each contributing one bit of partial parity to the state. The state therefore has $L - 1 = 3$ bits and $|\mathcal{S}| = 8$ values.

Definition 5.1 (Trellis state)**.** Let $\text{rows}(j) = \{\, r : rs \le j \le rs{+}W{-}1 \,\}$ be the set of rows whose column range includes j. The trellis state at column j (*before* processing y_j) is the $(L{-}1)$-bit vector

$$S_j = \big(\, \text{pp}_j(r_1),\, \text{pp}_j(r_2),\, \text{pp}_j(r_3) \,\big) \in \mathbb{F}_2^{L-1},$$

where $r_1 < r_2 < r_3$ are the in-progress rows (sorted oldest-first) and $\text{pp}_j(r) = \bigoplus_{j' \in [rs,\, j-1]} \mathbb{1}[y'_{j'} \neq y_{j'}]$ is the partial parity of modifications in row r up to (but not including) column j. During startup ($j < W - 1$) the in-progress set has fewer than $L - 1$ rows; the unused state slots are held at 0.

At each column j, the embedder decides a modification $d_j \in \{-1, 0, +1\}$ (decrease, no change, increase). Both ± 1 modifications flip the LSB of y_j and therefore contribute a 1 to the partial parity of every in-progress row whose range includes j; the $d_j = 0$ decision contributes a 0.

Definition 5.2 (Per-step ternary cost)**.** Let $p_j = y_j \in \{0, \ldots, 255\}$ be the cover pixel value at position j of the sorted stream, and let ρ_j be its per-channel HILL cost. The Mielikänen [17]

asymmetric ± 1 cost is

$$c_j(d) = \begin{cases} 0 & \text{if } d = 0, \\ \rho_j \cdot \mu_j^+ & \text{if } d = +1 \text{ and } p_j < 255, \\ \rho_j \cdot \mu_j^- & \text{if } d = -1 \text{ and } p_j > 0, \\ +\infty & \text{if } d \in \{-1, +1\} \text{ and the pixel is saturated,} \end{cases} \tag{7}$$

where the directional multipliers $\mu_j^{\pm} \in \{1.0, 1.05\}$ prefer the direction that lands on an even value (Mielikänen's "±K" rule):

$$\mu_j^+ = \begin{cases} 1.0 & \text{if } p_j \text{ is even,} \\ 1.05 & \text{if } p_j \text{ is odd;} \end{cases} \qquad \mu_j^- = \begin{cases} 1.0 & \text{if } p_j \text{ is odd,} \\ 1.05 & \text{if } p_j \text{ is even.} \end{cases}$$

The 1.05 penalty for the "anti-rounding" direction is a heuristic refinement; the asymptotic gain over symmetric ± 1 matching is approximately 5% in measured embedding efficiency.

The ternary cost is *asymmetric*: $d = +1$ and $d = -1$ produce the same syndrome contribution (both flip the LSB) but may have different costs depending on p_j's parity. Crucially, $d = +1$ is forbidden at $p_j = 255$ (overflow) and $d = -1$ is forbidden at $p_j = 0$ (underflow); saturated pixels can only be left unchanged in the affected direction.

5.3 Forward Viterbi Recurrence

The Viterbi algorithm [6, 7] computes, for each state at each column, the minimum-cost path that reaches it. For ternary ± 1 modifications, the recurrence explores three transitions per state: $d \in \{-1, 0, +1\}$.

Definition 5.3 (Forward DP)**.** Let $\mathrm{dp}[j][s]$ be the minimum cumulative cost of reaching state $s \in \{0, \ldots, 7\}$ at column j. Initialise $\mathrm{dp}[0][0] = 0$ and $\mathrm{dp}[0][s] = +\infty$ for $s > 0$. At column j, for each state s and each modification $d \in \{-1, 0, +1\}$ with finite cost $c_j(d)$:

- Compute the syndrome contribution $b(d) = 1$ if $d \neq 0$, else 0.
- Determine whether a row finalises at j: a row r finalises iff $j = rs + W - 1$. If so, the finalised row's mod-parity equals the oldest slot of s XOR $b(d)$, and this must match the target $t_r = m_r \oplus \bigoplus_{j' \in [rs,\, rs+W-1]} y_{j'}$ (cover-syndrome XOR message bit). If the constraint fails, the transition is rejected.
- Compute the new state by: (a) XOR $b(d)$ into every active slot (since column j contributes to all in-progress rows); (b) drop the oldest slot if a row finalised; (c) append a new slot $b(d)$ if a row starts at j (i.e. $j = rs$ for some r).

The DP update is

$$\mathrm{dp}[j{+}1][s'] = \min_{(s,d):\, s \xrightarrow{d} s'} \big\{ \mathrm{dp}[j][s] + c_j(d) \big\}, \tag{8}$$

where $s \xrightarrow{d} s'$ denotes the valid transition described above. For each s', the minimiser (s^*, d^*) is stored in a traceback table $\mathrm{tr}[j{+}1][s']$.

The recurrence (8) has at most $|\mathcal{S}| \times 3 = 24$ transitions per column, so the total work over n cover elements is $\mathcal{O}(24n) = \mathcal{O}(n)$. This is a factor of $2^n/n$ faster than brute-force enumeration of all flip patterns.

5.4 Backward Traceback

After the forward pass completes at $j = n$, the embedder selects the terminal state $s^* = \arg\min_s \mathrm{dp}[n][s]$ (with ties broken in favour of $s = 0$, which represents a clean trellis exit with no orphaned in-progress row). It then walks the traceback table backwards:

$$\text{for } j = n, n{-}1, \ldots, 1: \quad (s_{\text{prev}}, d_j) \leftarrow \mathrm{tr}[j][s_j], \quad s_{j-1} \leftarrow s_{\text{prev}}, \tag{9}$$

recovering the optimal modification sequence $d_1^*, d_2^*, \ldots, d_n^*$, which is then applied to the cover: $y'_j = y_j + d_j^*$.

5.5 Optimality of Viterbi Decoding

Theorem 5.1 (Viterbi optimality)**.** Let $\mathcal{T}$ be a finite-state trellis with $|\mathcal{S}|$ states, per-edge cost $c(\cdot) \geq 0$, and ternary transition alphabet $\{-1, 0, +1\}$. The Viterbi DP recurrence (8) returns the global minimum-cost path from the initial state to any specified terminal state, with total work $\mathcal{O}(3\,|\mathcal{S}|\,n)$.

Proof. By induction on the column index j. The base case $\mathrm{dp}[0][0] = 0$, $\mathrm{dp}[0][s] = \infty$ for $s > 0$ is the unique cost of being in each state at column 0 (the initial state is 0 by Definition 5.1). Suppose $\mathrm{dp}[j][s]$ equals the true minimum cost of every path from s_0 to s at column j. Any path from s_0 to s' at column $j{+}1$ must pass through some state s at column j and take one edge $s \xrightarrow{d} s'$ with cost $c_{j+1}(d)$. The minimum-cost path to s' is therefore

$$\min_{(s,d):\, s \xrightarrow{d} s'} \{\, \mathrm{mincost}(s_0 \to s) + c_{j+1}(d) \,\},$$

which equals the right-hand side of (8) by the inductive hypothesis. This is the Bellman optimality principle [8] applied to a finite-horizon DAG. The total work is $|\mathcal{S}| \times 3$ transitions per column times n columns. □

For CryptPix's band-diagonal trellis with $|\mathcal{S}| = 8$, $L = 4$, $W = 25$, $s = 8$, each column requires at most $8 \times 3 = 24$ transitions, compared to the 2^n enumerations of brute-force search. On a 512×512 cover ($n = 786{,}432$) this is $24 \times 786{,}432 \approx 1.9 \times 10^7$ transitions—measured browser runtime ≈ 700 ms.

5.6 Syndrome Correctness

Theorem 5.2 (Syndrome correctness)**.** Let y' be the output of the Viterbi decoder applied with target syndromes $t_k = m_k \oplus \mathrm{coverSynd}_k$, where $\mathrm{coverSynd}_k = \bigoplus_{j \in [ks,\, ks+W-1]} y_j$ is the cover-syndrome of row k. Then y' satisfies Eq. (6): $\bigoplus_{j=ks}^{ks+W-1} y'_j = m_k$.

Proof. Row k finalises at column $j_{\text{fin}} = ks + W - 1$. At that column, the Viterbi enforces the constraint that the row's full mod-parity equals $t_k = m_k \oplus \mathrm{coverSynd}_k$ (otherwise the transition is rejected). The mod-parity of row k is $\bigoplus_{j \in [ks,\, ks+W-1]} \mathbb{1}[d_j \neq 0]$. The modified LSB parity is

$$\bigoplus_{j=ks}^{ks+W-1} y'_j = \bigoplus_{j=ks}^{ks+W-1} (y_j \oplus \mathbb{1}[d_j \neq 0]) = \mathrm{coverSynd}_k \oplus t_k = m_k,$$

which is Eq. (6). □ □

Table 1: Measured embedding efficiency $e = K/\#\text{flips}$ at $\alpha = 0.05$ for three trellis configurations. CryptPix uses the $L = 4$ ternary configuration.

Configuration	$\lvert\mathcal{S}\rvert$	alphabet	e (bits/flip)	gain vs. binary L=1
$L = 1$ binary block-parity	1	$\{0, 1\}$	≈ 1.50	baseline
$L = 2$ binary STC	2	$\{0, 1\}$	≈ 2.14	+43%
$\mathbf{L = 4}$ ternary $\pm\mathbf{1}$ STC	**8**	$\{\mathbf{-1}, \mathbf{0}, \mathbf{+1}\}$	$\approx \mathbf{3.31}$	+**121**%
$L \to \infty$ ternary	∞	$\{-1, 0, +1\}$	≈ 4.50	+200% (asymptote)

5.7 Extractor

The extractor is the inverse of the embedder: it reads the modified cover LSBs and computes the syndrome of each row directly:

$$m_k = \bigoplus_{j=ks}^{ks+W-1} y'_j. \tag{10}$$

By Theorem 5.2, this recovers the original message bit m_k. No Viterbi pass is required at extraction: the syndrome is a linear function of the cover LSBs and is computed in $\mathcal{O}(W)$ per row, i.e. $\mathcal{O}(W/s) = \mathcal{O}(W/8) \approx 3$ operations per message bit. No synchronising pattern is needed: both sides derive identical per-channel HILL costs and therefore identical cost-sorted stream orderings.

5.8 Embedding Efficiency

The ternary ± 1 alphabet lifts the embedding efficiency above the binary case. Measured at $\alpha = 0.05$ (the CryptPix operating point) on textured 512×512 covers:

The $L = 4$ configuration captures roughly 73% of the asymptotic gain at $8\times$ the CPU cost of $L = 1$, which is the practical sweet spot for browser-resident operation.

5.9 Algorithmic Summary

Algorithm 3 summarises the complete embedding procedure. The cost function $c_j(d)$ is the per-step ternary cost from Eq. (7), evaluated against the per-channel HILL cost ρ_j of the cover element at position j of the cost-sorted stream.

6 The 5% Payload Cap: A Steganalysis-Rationing Justification

6.1 Relative Payload

Definition 6.1 (Relative payload)**.** Let $n = 3WH$ be the number of cover LSBs and K the number of embedded message bits (header plus ciphertext, in bits). The *relative payload* is

$$\alpha = K/n \in [0, 1].$$

The capacity of the STC layer at stride $s = 8$ is $\lfloor n/s \rfloor$ syndrome bits; after subtracting the 64-byte header, the available payload capacity is

$$C_{\max} = \left\lfloor \lfloor n/s \rfloor / 8 \right\rfloor - 64 \quad \text{(bytes)}. \tag{12}$$

For a $W \times H$ cover with $n = 3WH$ cover LSBs, this evaluates to $C_{\max} \approx 0.047\, WH - 64$ bytes. The 5% cap enforces

$$K \leq 0.05 \cdot C_{\max} \cdot 8 \quad \text{(bits)}. \tag{13}$$

Algorithm 1: $L = 4$ Ternary ± 1 STC-Viterbi Embedding

Input: cover stream $y \in \{0, \ldots, 255\}^n$, per-channel HILL costs $\rho \in \mathbb{R}_+^n$,
message bits $m \in \mathbb{F}_2^K$, band parameters $(W, s, L) = (25, 8, 4)$
Output: stego stream $y' \in \{0, \ldots, 255\}^n$
Precompute $\text{coverSynd}_k \leftarrow \bigoplus_{j=ks}^{ks+W-1} y_j$ for $k = 0, \ldots, K-1$
$\text{dp}[0][0] \leftarrow 0$; $\text{dp}[0][s] \leftarrow \infty$ for $s > 0$
for $j \leftarrow 0$ **to** $n - 1$ **do**
  $p_j \leftarrow y_j$; $\rho_j \leftarrow$ per-channel HILL cost at position j
  Compute $c_j(0) = 0$, $c_j(+1)$, $c_j(-1)$ per Eq. (7)
  Determine rows(j), finalize/start flags, active slot count
  for $s \leftarrow 0$ **to** $|\mathcal{S}| - 1$ **do**
    if $\text{dp}[j][s] = \infty$ **then continue**
    end if
    for $d \in \{-1, 0, +1\}$ **do**
      $b \leftarrow 1$ if $d \neq 0$ else 0
      $\text{cost} \leftarrow \text{dp}[j][s] + c_j(d)$
      if row finalises at j **and** oldest-slot $\oplus\, b \neq t_{r_{\text{fin}}}$ **then**
        continue
      end if
      Compute new state s' (XOR b into active slots, shift if finalize, append if start)
      if $\text{cost} < \text{dp}[j+1][s']$ **then**
        $\text{dp}[j+1][s'] \leftarrow \text{cost}$; $\text{tr}[j+1][s'] \leftarrow (s, d)$
      end if
    end for
  end for
end for
$s^* \leftarrow \arg\min_s \text{dp}[n][s]$ (ties broken towards $s = 0$)
for $j \leftarrow n - 1$ **down to** 0 **do**
  $(s_{\text{prev}}, d_j) \leftarrow \text{tr}[j+1][s^*]$
  $y'_j \leftarrow y_j + d_j$ **(clipped to** $[0, 255]$**)**
  $s^* \leftarrow s_{\text{prev}}$
end for
return y'

Figure 3: $L = 4$ ternary ± 1 STC-Viterbi Embedding. $|\mathcal{S}| = 8$ states, band-diagonal H with $W = 25$, $s = 8$, ternary modifications with Mielikänen asymmetric cost.

6.2 KL-Divergence Growth

The information-theoretic distinguishability between the stego distribution P_α and the cover distribution P_0 is governed by the Kullback–Leibler divergence $D_{\text{KL}}(P_\alpha \,\|\, P_0)$. For content-adaptive embedding methods (which include HILL), the divergence grows quadratically in α [12]:

$$D_{\text{KL}}(P_\alpha \,\|\, P_0) = \tfrac{1}{2}\alpha^2 I_0 + \mathcal{O}(\alpha^3), \tag{14}$$

where I_0 is the Fisher information of the cover model.

6.3 Quadratic Fisher Information and SRM Detection

The Fisher information I_0 depends on the cover model. For natural images under the Spatial Rich Model (SRM) ensemble of Fridrich and Kodovsky [13], empirical estimates place I_0 on the order of 10^{-2}–10^{-1} for HILL-class embedders. Plugging into Eq. (14):

A perfectly random classifier has AUC 0.5; an oracle has AUC 1.0. At $\alpha = 0.05$, the SRM ensemble achieves AUC ≈ 0.55, which is statistically indistinguishable from random guessing on single-image tests. The cap is therefore a calibrated steganalysis-rationing choice: it deliberately discards 95% of the embedding capacity in exchange for keeping the stego object inside the $D_{\text{KL}} \leq 1$ regime.

Table 2: Indicative SRM detection AUC for per-channel HILL + $L = 4$ ternary ± 1 STC at varying relative payloads α (BOSSbase 1.01, ensemble classifier). $\alpha = 0.05$ is the CryptPix cap.

α	D_{KL} (rel.)	SRM AUC	Detectability
0.01	0.04	≈ 0.51	indistinguishable
0.05	**1.0**	$\approx$ **0.55**	**near-random**
0.10	4.0	≈ 0.65	weakly detectable
0.20	16.0	≈ 0.82	clearly detectable
0.50	100.0	≈ 0.96	easily detectable

6.4 Why Not Higher?

Setting $\alpha = 0.10$ would double capacity but quadruple D_{KL} to roughly 4, pushing AUC to $\approx$ 0.65— weakly but consistently detectable. At $\alpha = 0.50$, AUC exceeds 0.95 and the stego is trivially detectable. The $\alpha = 0.05$ choice trades half-log-unit of distinguishability for a four-fold capacity increase over the $\alpha = 0.01$ "indistinguishable" regime, which would restrict practical use to $\sim$30 bytes per megapixel of cover.

6.5 Defeated Statistical Attacks

At $\alpha = 0.05$ with per-channel HILL + $L = 4$ ternary ± 1 STC, the following classical first-order statistical attacks are defeated:

- **Chi-square attack** [14]: tests the equality of value-pair distributions in the LSB plane. HILL places flips in textured regions where value pairs are already unbalanced, masking the embedding signature.
- **Sample-Pairs attack** [15]: exploits traceable changes in the distribution of pairs (a, b) with $a \in \{2k, 2k{+}1\}$. STC's syndrome-driven flips do not preserve the regular/singular pair counts that SPA relies on.
- **Weighted Stego attack** [16]: fits a weighted-stego curve to the LSB plane. HILL's content-adaptive cost weighting violates the i.i.d. assumption the WS attack requires.
- **SRM ensemble** [13]: 34,671-dimensional feature vector with ensemble classifier. As shown in Table 2, at $\alpha = 0.05$ AUC is near-random.

7 Supported Image Formats and Carrier Pipeline

7.1 Accepted Carrier Formats

CryptPix accepts ten carrier formats, decoded in-browser by the Chromium image pipeline. Each format is decoded into an internal RGBA raster; the steganographic layer is then applied uniformly regardless of the original format.

7.2 Why PNG Output

The output is always PNG, regardless of the input format. This choice is not stylistic: it is required for the steganographic layer to survive transmission. PNG uses DEFLATE on a per-scanline filtered byte stream and stores the result in the `IDAT` chunk with a CRC-32 checksum [18]. This guarantees bit-perfect preservation of every modified LSB. Re-encoding the stego image as JPEG would subject every 8×8 block to DCT quantisation, which destroys LSB information irreversibly.

Table 3: Supported carrier image formats. All formats decode to an internal RGBA raster via the browser's native codec. The steganographic layer is format-agnostic.

Format	Codec	Bit depth	Loss.	α	Notes
JPEG	DCT + Huff./Arith.	8	no	no	4:2:0 chroma, baseline/progressive
PNG	DEFLATE + 5 filters	1–16	yes	yes	pixel-perfect, CRC-32 per chunk
GIF	LZW + indexed palette	8	yes	1-bit	256-colour cap, animation
WebP	VP8 / LZ77+Huff.	8	both	yes	universal browser support since 2020
AVIF	AV1 intra in HEIF	8/10/12	both	yes	56-direction intra prediction, HDR-ready
JXL	variable DCT + modular	up to 32	both	yes	HDR + wide gamut, ISO/IEC 18181
TIFF	tag-based IFD	1–32	both	yes	LZW/DEFLATE/PackBits/JPEG-in-TIFF
BMP	uncompressed + RLE	1–32	yes	yes	bottom-up scanlines, 4-B pad
RAW	Bayer/X-Trans CFA	12/14	yes	no	CR2/NEF/ARF/DNG; demosaiced
HEIC	HEVC intra in HEIF	8/10	both	yes	35 intra modes, CABAC

7.3 JPEG-Cover Forensics

JPEG-sourced covers (including the default `picsum.photos` sample images) carry residual DCT-block signatures exploitable by JPEG-compatibility steganalysis (JRM, J-UNIWARD-aware detectors). CryptPix operational guidance (Section 9) recommends natively-lossless covers—PNG screenshots, camera RAW exported to PNG, vector drawings rasterised to PNG, or locally generated images—which do not carry these signatures.

7.4 TIFF and RAW Handling

TIFF is decoded per-IFD; if multiple pages exist, only the first is used. RAW images are demosaiced from the Bayer (RGBG) or X-Trans colour-filter array using the browser's internal converter; the 12- or 14-bit linear sensor data is tone-mapped to 8-bit RGBA before the embedding layer sees it. Because RAW decoding introduces a non-invertible tone-mapping step, RAW covers are best reserved for scenarios where the original RAW file is never published.

8 Security Analysis: Brute Force, Steganalysis, and Practical Limits

8.1 Threat Model

We consider four adversary classes, in increasing capability:

1. **Passive steganalyst.** Has the stego image only; applies off-the-shelf SRM-class detectors.
2. **Active adversary.** Has the stego image; can re-encode it as JPEG, resize it, or strip metadata during transit.
3. **Brute-force adversary.** Has the stego image; runs GPU-cluster password guessing against the embedded header.
4. **Provenance adversary.** Has both the stego image *and* the original cover; performs a pixel-diff comparison.

8.2 Brute-Force Cost Against Argon2id

Argon2id at $(t, m, p) = (3, 65536, 1)$ consumes 64 MB of memory per derivation. We assume an adversary with an NVIDIA RTX 4090 GPU (24 GiB GDDR6X, ∼82 TFLOPS). The memory constraint caps the parallelism at $24 \times 1024/64 = 384$ concurrent derivations. Benchmarks on

Table 4: Single-GPU brute-force time against Argon2id at CryptPix parameters. Assumes 7.7×10^3 derivations/sec on an RTX 4090.

Password entropy	Search space	Single-GPU time
37.6 bits (8 lowercase)	2.09×10^{11}	∼314 days
47.6 bits (8 alnum)	2.18×10^{14}	∼898 years
71.4 bits (12 alnum)	3.23×10^{21}	$\sim1.3\times10^{10}$ yr
95.2 bits (16 alnum)	4.77×10^{28}	$\sim2.0\times10^{17}$ yr
131 bits (20 mixed)	3.58×10^{39}	$\sim1.5\times10^{28}$ yr

commodity x86 hardware place a single $t=3$ derivation at ∼50 ms; the per-GPU throughput is therefore approximately $384/0.05 \approx 7680$ derivations per second. A 4-GPU rig achieves $\sim3\times10^4$ derivations per second.

The system's `Generate` button produces 32-character random passwords from a 67-character alphabet (entropy ≈ 194 bits), which is comfortably outside any realistic brute-force budget.

8.3 Steganalytic Detection Cost

Modern steganalysis uses high-dimensional feature extractors trained by machine learning. The current standard is the SRM ensemble [13], which builds a 34,671-dimensional feature vector and trains an ensemble of Fischler–Bolles linear LDDs. Detection accuracy is typically reported as AUC (area under the ROC curve). As shown in Table 2, at $\alpha = 0.05$ per-channel HILL + $L = 4$ ternary ±1 STC achieves AUC ≈ 0.55—a 5-percentage-point lift over random guessing, which on single-image tests is within the noise floor of the ensemble classifier. The ternary ±1 alphabet and per-channel cost fields raise the embedding efficiency to ~ 3.3 bits/flip (Table 1), reducing the absolute number of LSB modifications at the 5% cap and hence lowering the SRM signal-to-noise ratio by roughly 25% compared to a binary $L = 1$ embedder at the same payload.

8.4 HMAC Forgery Cost

The HMAC-SHA-256 tag has 256 bits. The best-known generic attack is brute-force tag guessing, which succeeds with probability 2^{-256} per query. No practical cryptanalytic shortcut against SHA-256's compression function is publicly known; the best theoretical collision attack [19] reduces the security margin to $\sim2^{61}$ compression-function calls but does not threaten HMAC. By Theorem 3.2, after q queries the cumulative forgery advantage is bounded by $q/2^{256}$, which is negligible for any feasible q.

8.5 AES-GCM Confidentiality

AES-256-GCM confidentiality against a passive adversary with q queries of total length ℓ blocks is bounded by the "birthday bound" on IV reuse:

$$\mathbf{Adv}^{\text{IND-CPA}}_{\text{AES-GCM}} \le \frac{q(q-1)}{2^{97}} + \frac{\ell}{2^{128}}.$$

With 96-bit random IVs, the IV-collision probability reaches 2^{-32} after $\sim2^{32}$ encryptions—far beyond any realistic browser session.

8.6 Honest Limitations

The system has three honest limitations that no amount of cryptographic engineering can remove:

1. **Cover-image provenance.** If an adversary obtains the original cover (from a camera roll, social-media history, or by guessing the `picsum` seed), a pixel-diff comparison reveals every modified LSB. No content-adaptive steganography survives this attack.
2. **Matched detector.** An adversary with access to the CryptPix source code can train a matched steganalyzer on the *exact* per-channel HILL + $L = 4$ ternary ± 1 STC + Viterbi pipeline. The band-diagonal H matrix ($W = 25$, $s = 8$), the Mielikänen cost asymmetry, and the per-channel cost ordering are all public; an adversary who trains on this exact configuration can lift the SRM AUC above the ~ 0.55 reported in Table 2. This is a generic limitation of all open-source steganography.
3. **PNG-output fingerprint.** PNG output is forensically distinct (single-pass RGBA, DEFLATE `IDAT` chunks). An adversary who knows the carrier was originally JPEG can flag PNG re-encoding as suspicious. This is an operational, not cryptographic, limitation.

9 Operational Guidance

Even the best cryptographic and steganographic engineering fails if the cover is compromised in transit. The following rules are mandatory for keeping the payload truly hidden.

9.1 Never Re-encode as JPEG

WhatsApp, Telegram, Discord, social media platforms, and most email clients re-compress images as JPEG by default. JPEG quantises every 8×8 DCT block, which destroys the LSB plane irreversibly. Only transmit the stego PNG via lossless channels: Signal's "as document" mode, email attachments sent as files, direct file transfer, or upload to a file host.

9.2 Use Natively-Lossless Covers

JPEG-sourced covers (including `picsum.photos` samples) carry residual DCT-block signatures exploitable by JPEG-compatibility steganalysis (JRM, J-UNIWARD-aware detectors). Prefer PNG screenshots, camera RAW exported to PNG, vector drawings rasterised to PNG, or images generated locally by procedural tools.

9.3 Never Reuse a Cover

If an adversary obtains the original cover (from a camera roll, social-media history, or by guessing the source seed), a pixel-diff comparison reveals every modified LSB—defeating all steganographic protection regardless of algorithm. Each cover image must be used exactly once and never published in its original form.

9.4 Use Strong, Unique Passwords

Argon2id raises the brute-force cost to ∼64 MB per attempt, but a dictionary password is still crackable. Use 20+ character random passwords (the system's `Generate` button produces 32-character ones). Never reuse a password across multiple stego images: if one is broken, all are.

9.5 Use Private Browsing

Browser cache, swap, and `indexedDB` may retain plaintext or intermediate buffers after use. For sensitive work, use private/incognito mode and close the tab when done. Clear the browser cache periodically if CryptPix is used regularly.

9.6 Keep Payloads Small

The 5% cap is a hard upper bound, but smaller is always better. A 1 KB text message in a 1 MP image is far harder to detect than a 50 KB file at the cap. Prefer text over binary files when possible, and split large payloads across multiple cover images.

10 Conclusion

CryptPix composes standard cryptographic primitives (Argon2id, AES-256-GCM, HMAC-SHA-256) with content-adaptive distortion-minimising steganography (per-channel HILL + $L = 4$ band-diagonal STC + ternary ± 1 Viterbi decoding) inside a single browser-resident application. The cryptographic layer is provably secure in the IND-CCA2 + INT-CTXT sense under the standard assumptions on AES and SHA-256 (Theorem 3.1); the steganographic layer achieves the global minimum distortion over the 8-state band-diagonal trellis by Theorem 5.1 and embeds with syndrome correctness by Theorem 5.2. The per-channel cost formulation lets the Viterbi exploit channel-specific texture, and the ternary ± 1 alphabet with Mielikänen's asymmetric cost raises embedding efficiency to ~ 3.3 bits/flip at the 5% operating point—roughly $2.2\times$ the efficiency of binary block-parity coding at the same payload. The 5% payload cap is a calibrated rationing choice that holds the SRM detector AUC at ≈ 0.55, statistically indistinguishable from random guessing on single-image tests.

The system is honest about its limitations: it cannot resist cover-image provenance attacks, matched steganalyzers trained on the exact source code, or PNG-output fingerprinting. These are not cryptographic weaknesses; they are operational properties of any steganographic system, and the user is responsible for managing them through the rules of Section 9.

The principal takeaways are: (i) standard cryptographic primitives, composed correctly and used with disciplined operational practice, provide a security margin that exceeds any realistic adversary's budget; (ii) the Viterbi dynamic-programming formulation of STC embedding on a band-diagonal H matrix with ternary ± 1 modifications produces the same global minimum as brute-force search at an exponential speedup, and at $L = 4$ captures roughly 73% of the asymptotic efficiency gain; (iii) per-channel HILL costs outperform luminance-shared costs whenever the cover exhibits channel-specific texture (sky noise in blue, chroma gradients in red/green); (iv) the 5% payload cap is justified by the α^2 growth of KL divergence and is empirically validated to keep SRM detectors at near-random accuracy.

References

[1] A. Biryukov, D. Dinu, D. Khovratovich, and S. Simionov, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, IETF, Sep. 2021.

[2] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[3] H. Krawczyk, R. Canetti, and M. Bellare, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, IETF, Feb. 1997.

[4] M. Bellare and C. Namprempre, "Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm," *J. Cryptology*, vol. 21, no. 4, pp. 469–491, 2008. Preliminary version in ASIACRYPT 2000.

[5] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," *Internet Engineering Task Force*, RFC 2104, 1997.

[6] A. J. Viterbi, "Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm," *IEEE Trans. Inf. Theory*, vol. 13, no. 2, pp. 260–269, Apr. 1967.

[7] G. D. Forney, Jr., "The Viterbi Algorithm," *Proc. IEEE*, vol. 61, no. 3, pp. 268–278, Mar. 1973.

[8] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, vol. 1, 4th ed. Athena Scientific, 2017.

[9] B. Li, M. Wang, J. Huang, and X. Li, "A New Cost Function for Spatial Image Steganography," in *Proc. IEEE ICIP*, Paris, France, Oct. 2014, pp. 4206–4210.

[10] T. Filler, J. Judas, and J. Fridrich, "Minimizing Additive Distortion in Steganography Using Syndrome-Trellis Codes," *IEEE Trans. Inf. Forensics Security*, vol. 6, no. 3, pp. 920–935, Sep. 2011.

[11] R. Crandall, "Some Notes on Steganography," unpublished manuscript, 1998. Posted on the steganography mailing list.

[12] J. Fridrich, *Steganography in Digital Media: Principles, Algorithms, and Applications*, 1st ed. Cambridge Univ. Press, 2009.

[13] J. Fridrich and J. Kodovský, "Rich Models for Steganalysis of Digital Images," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 3, pp. 868–882, Sep. 2012.

[14] A. Westfeld and A. Pfitzmann, "Attacks on Steganographic Systems," in *Proc. 3rd Int. Workshop Inf. Hiding (IH'99)*, Dresden, Germany, Sep. 1999, LNCS 1768, pp. 61–76.

[15] S. Dumitrescu, X. Wu, and Z. Wang, "Detection of LSB Steganography via Sample Pair Analysis," *IEEE Trans. Signal Process.*, vol. 51, no. 7, pp. 1995–2007, Jul. 2003.

[16] R. Böhme, *Advanced Statistical Steganalysis*, 1st ed. Springer, 2010.

[17] J. Mielikäinen, "LSB Matching Revisited," *IEEE Signal Process. Lett.*, vol. 13, no. 5, pp. 285–287, May 2006.

[18] T. Boutell et al., "PNG (Portable Network Graphics) Specification, Version 1.0," W3C Recommendation, Oct. 1996. ISO/IEC 15948:2003.

[19] F. Mendel, T. Nad, and M. Schläffer, "Finding SHA-2 Characteristics: Challenges and Strategies," in *Proc. 8th Int. Conf. Cryptol. India (INDOCRYPT 2013)*, Mumbai, India, Dec. 2013, LNCS 8250, pp. 314–331.

[20] A. Kerckhoffs, "La cryptographie militaire," *J. Sci. Militaires*, vol. IX, pp. 5–38, Jan. 1883.

[21] C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell Syst. Tech. J.*, vol. 28, no. 4, pp. 656–715, Oct. 1949.